

Integrantes

Alberto Giordano Gomez Gonzalez – 4.862.694-9

Valentin Rodriguez Pintos – 5.193.955-9

Guadalupe Sosa Acosta – 5.660.478-1

1. Objetivos

El objetivo general de este ejercicio es desarrollar un sistema concurrente en Python que permita el envío dinámico de tareas entre clientes y su procesamiento concurrente en un servidor central. A través de este ejercicio, se busca familiarizarse con los conceptos de hilos, semáforos y locks, así como desarrollar habilidades en programación concurrente.

De los objetivos puntuales, destacan los siguientes.

- Implementar una solución que permita el envío de tareas desde múltiples clientes hacia un servidor central.
- Utilizar hilos para lograr la concurrencia en el procesamiento de las tareas
- Emplear semáforos para limitar la cantidad de hilos concurrentes en el servidor.
- Utilizar locks para garantizar la exclusión mutua al acceder a los archivos de resultados.

2. Material

- Computadora con interpretador Python v3.9 o superior e IDE/editor de código-fuente.
- Librerías nativas de Python y las complementarias threading, time, logging y sympy.

3. Fundamentación teórica

3.1. Programación Orientada a Objetos (OOP)

La Programación Orientada a Objetos es un paradigma que organiza el software en torno a objetos que combinan estado (atributos) y comportamiento (métodos), con el propósito de mejorar la modularidad, la reutilización y la mantenibilidad del código. Este enfoque permite diseñar componentes que se adaptan y se combinan sin rehacer el sistema entero, favoreciendo el desarrollo de sistemas complejos y escalables. (Morero, 2000)

La adopción de OOP aporta ventajas prácticas: reutilización de código mediante clases, facilidad para extender funcionalidad por herencia y, crucialmente, la protección de la implementación a través de la encapsulación. Esta estructura modular simplifica el mantenimiento y se refleja en la implementación concreta en lenguajes modernos. (Morero, 2000).

3.2. Programación Concurrente en Python

La programación concurrente busca que un programa pueda realizar varias tareas 'al mismo tiempo', aunque no necesariamente se ejecuten todas de forma simultánea. Lo que sí sucede es que las tareas se intercalan, lo que permite aprovechar mejor los recursos del sistema y dar al usuario una sensación de mayor fluidez (Downey, 2012).

3.3. Hilos en Python: Módulo `threading`

El módulo `threading` es la herramienta principal de Python para la programación concurrente basada en hilos. Permite que un solo proceso ejecute múltiples hilos de ejecución, los cuales operan de forma intercalada (concurrente) en lugar de simultánea (paralela).

La característica fundamental de `threading` es que todos los hilos creados comparten el mismo espacio de memoria que el proceso padre. Esto reduce el uso de recursos y facilita la comunicación entre tareas. Sin embargo, esta arquitectura introduce un desafío crítico: el riesgo de condiciones de carrera (*race conditions*), que ocurren cuando varios hilos intentan leer y modificar un mismo dato al mismo tiempo, lo que puede corromper la información. (Python Software Foundation, 2024)

3.4. Global Interpreter Lock (GIL)

El GIL (Global Interpreter Lock) es un mecanismo interno de la implementación CPython. Es un bloqueo por proceso que impide que más de un hilo (*thread*) ejecute código Python al mismo tiempo, incluso en procesadores con múltiples núcleos (Python Software Foundation, 2024).

Esto tiene dos implicancias distintas. En el caso de `multiprocessing`, al crear un proceso nuevo, este obtiene su propia instancia del intérprete y, por lo tanto, su propio GIL. Como los procesos no compiten por el mismo bloqueo, el sistema operativo puede asignarlos a diferentes núcleos de CPU y ejecutarlos en paralelo real.

En cambio, con `threading`, todos los hilos comparten el mismo proceso y, por ende, compiten por el único GIL. Esto significa que no pueden ejecutarse simultáneamente si están realizando operaciones intensivas en CPU (*CPU-bound*). Los hilos se ven forzados a ejecutarse de forma concurrente (turnándose) en lugar de paralela, lo que limita la aceleración (*speedup*) que se puede obtener.

3.5. Tareas Ligadas a CPU vs. Tareas Ligadas a E/S (CPU-Bound vs. I/O-Bound)

Para comprender el impacto real del GIL y la utilidad de `threading`, es crucial diferenciar dos tipos de cargas de trabajo:

- **Tareas Ligadas a CPU (CPU-Bound):** Son tareas que pasan la mayor parte de su tiempo realizando cálculos. Están limitadas por la velocidad del procesador.
- **Tareas Ligadas a E/S (I/O-Bound):** Son tareas que pasan la mayor parte de su tiempo esperando que se completen operaciones de Entrada/Salida (E/S), como leer un archivo, escribir en un log, o esperar una respuesta de red.

El GIL es un cuello de botella significativo para tareas puramente *CPU-Bound*. Sin embargo, la implementación de Python está diseñada para que un hilo libere el GIL mientras está bloqueado esperando que termine una operación de E/S. En el instante en que un hilo se bloquea, el GIL se libera, permitiendo que otro hilo lo adquiera y comience su propia ejecución. Cuando la operación de E/S termina, el primer hilo vuelve a la cola para readquirir el GIL y continuar (Python Software Foundation, 2024).

3.6. Desafíos de la Concurrencia: Secciones Críticas y Condiciones de Carrera

Cuando múltiples hilos comparten el mismo espacio de memoria, surge el desafío de proteger los recursos compartidos. Una **sección crítica** es el segmento de código que accede y modifica un recurso compartido (como una variable, un contador o un archivo).

Si dos o más hilos intentan ejecutar esa sección crítica al mismo tiempo, y el resultado de la operación depende del orden impredecible en que se ejecuten, ocurre una **condición de carrera** (*race condition*). Este es uno de los errores más difíciles de depurar en la programación concurrente, ya que puede llevar a resultados incorrectos, inconsistentes o a la corrupción total de los datos (Silberschatz et al., 2018).

3.7. Mecanismos de Sincronización

Para prevenir las condiciones de carrera y coordinar la ejecución de los hilos, se utilizan mecanismos de sincronización. Estos mecanismos aseguran que el acceso a las secciones críticas se realice de manera ordenada y segura.

3.7.1. Locks y Exclusión Mutua

La solución directa a una condición de carrera es imponer la **exclusión mutua**. Este principio garantiza que, si un hilo está ejecutando una sección crítica, ningún otro hilo pueda entrar en ella al mismo tiempo.

La implementación más común de este principio es el **Mutex Lock** (o cerrojo). Un *Lock* tiene dos estados: "adquirido" (locked) y "liberado" (unlocked). Antes de entrar a la sección crítica, el hilo debe "adquirir" el *Lock*. Si el *Lock* ya está adquirido por otro hilo, el hilo solicitante se bloquea y debe esperar. Al salir de la sección crítica, el hilo "libera" el *Lock*, permitiendo que otro hilo en espera pueda adquirirlo (Silberschatz et al., 2018).

3.7.2. Semáforos

Mientras que un *Lock* impone exclusión total (solo un hilo a la vez), un **Semáforo** es un mecanismo más general. Un semáforo mantiene un contador interno (N) que gestiona el acceso a un número limitado de recursos o permite que un número específico de hilos accedan a una sección (Silberschatz et al., 2018).

En lugar de una exclusión estricta (N=1), un semáforo con un valor N permite que hasta N hilos adquieran el semáforo y entren en la sección simultáneamente. El hilo N+1 que intente adquirirlo se bloqueará hasta que uno de los N hilos anteriores libere el semáforo. Esta herramienta es fundamental para controlar el **grado de concurrencia**, permitiendo limitar la carga del sistema y evitar la saturación de recursos.

3.8. Computación Simbólica: La librería SymPy

La computación simbólica se ocupa del cómputo de objetos matemáticos de forma simbólica. Esto significa que los objetos matemáticos se representan de forma exacta, no aproximada, y las expresiones matemáticas con variables no evaluadas se dejan en forma simbólica. **SymPy** es una librería de Python para realizar esta álgebra simbólica, permitiendo trabajar con expresiones matemáticas como objetos, hacer derivadas, integrales, simplificaciones, entre otras operaciones (SymPy Developers, 2024).

3.9. Registro de Eventos: El módulo logging

El módulo **logging** es la herramienta estándar de Python para el registro de eventos. Ofrece un sistema flexible para emitir mensajes de seguimiento sobre el estado y la ejecución de una aplicación.

A diferencia del uso de sentencias `print()`, el módulo **logging** permite configurar diferentes **niveles de severidad** (como DEBUG, INFO, WARNING, ERROR) y dirigir los mensajes a múltiples destinos, como la consola o archivos de registro persistentes (Python Software Foundation, 2024).

En el contexto de la programación concurrente, un registro de eventos robusto es fundamental. Permite rastrear la intercalación y el comportamiento de los hilos, y mediante la inclusión de marcas de tiempo (*timestamps*), se convierte en una herramienta esencial para la **medición de rendimiento** y la depuración, permitiendo analizar la duración de tareas individuales.

4. Metodología

Para cumplir con los objetivos del proyecto, es decir, implementar un sistema concurrente funcional y, a la vez, medir su rendimiento y validar su correctitud, se diseñó la siguiente metodología experimental.

El procedimiento utiliza el script de simulación (ver Apéndice A) para procesar un conjunto de tareas de entrada estandarizado (ver Apéndice B). La metodología se centra en ejecutar este script bajo diferentes configuraciones para aislar y medir el impacto del semáforo (conurrencia) y de los locks (sincronización).

Los datos recolectados en cada fase experimental son los que se utilizarán para completar las tablas de resultados.

4.1. Descripción de la Implementación

1. **Clase BaseDeDatos:** Encapsula el semáforo que limita la concurrencia (`max_hilos_concurrentes`) y los Lock para asegurar la exclusión mutua al escribir en los archivos de resultados.
2. **Función procesar_tarea:** Es el núcleo del trabajador (`target`) de cada hilo. Procesa tareas individuales (integral, derivada, tesis), adquiriendo primero el semáforo y luego utilizando locks específicos para proteger la escritura en archivos.
3. **Gestión de la Demanda:** El script primero asegura la existencia del archivo de entrada (`demanda.txt`, Apéndice B). Si el archivo no existe, la función `crear_demanda_si_no_existe` lo genera automáticamente con un conjunto de datos de ejemplo. Inmediatamente después, `leer_tareas_demanda` parsea este archivo (con formato `tipo:datos`) y genera la lista de tareas a procesar.
4. **Funciones de Experimentación (main):** El script incluye la lógica para ejecutar los experimentos, iterando sobre los valores del semáforo (`VALORES_N_SEMAFORO`) y recopilando el tiempo total y el valor final de `largo.txt`. Permite deshabilitar los locks (`use_locks=False`) para pruebas de condición de carrera.
5. **Funciones de Análisis (analizar_log):** El módulo logging captura los eventos de duración. Un analizador extrae estos datos del log para generar estadísticas de rendimiento por tipo de tarea.

4.2. Procedimiento Experimental

Sobre la base de esta implementación, el procedimiento para la recolección de datos se divide en las siguientes fases:

4.2.1. Establecimiento de la Línea Base Secuencial

Para poder cuantificar la "mejora" (Speedup) que introduce la concurrencia, es imprescindible primero establecer una línea base de rendimiento.

1. Se ejecuta el script configurando el semáforo con $N=1$. Esto fuerza al sistema a operar de forma secuencial (una tarea a la vez).
2. Se mide el tiempo total de esta ejecución. Este dato se usará como el tiempo de referencia (1.00x) en las Tablas 1 y 2.
3. Simultáneamente, esta corrida genera un resultado garantizado en `largo.txt` (ya que no hay condiciones de carrera). Este valor se anota como el "Valor Esperado (Correcto)", que servirá para todas las verificaciones de integridad en la Tabla 5.

4.2.2. Pruebas de Escalabilidad (Impacto del Semáforo)

El objetivo de esta fase es medir cómo escala el rendimiento del sistema al aumentar el número de hilos concurrentes permitidos por el semáforo.

1. Se ejecuta el script en un bucle, repitiendo la prueba completa para los diferentes niveles de concurrencia ($N = 5, 10, 20$ y 50).
2. Para cada valor de N , se registra el tiempo total de ejecución y el valor final de largo.txt.
3. Los datos de tiempo permiten calcular el Speedup ($\text{Tiempo}(N=1) / \text{Tiempo}(N)$) y así completar la Tabla 2.
4. Los valores de largo.txt obtenidos se comparan contra el "Valor Esperado" de la Fase 1 para verificar la integridad de los datos en la Tabla 5.

4.2.3. Validación de Sincronización (Impacto de Locks)

Esta prueba busca demostrar empíricamente por qué los locks son necesarios y, a su vez, medir su overhead (costo en rendimiento). Para esto, se realizan dos pruebas comparativas con un N fijo ($N=10$):

1. **Con Locks:** Se utiliza el resultado de tiempo y de integridad de largo.txt ya obtenidos en la Fase 2 para $N=10$.
2. **Sin Locks:** Se ejecuta el script una vez más, deshabilitando explícitamente los locks (`use_locks=False`). Se registra su tiempo total y el valor resultante de largo.txt.

Los resultados de ambos se usan para llenar la Tabla 3, contrastando el tiempo de ejecución y la correctitud del resultado.

4.2.4. Análisis Granular de Tareas

Finalmente, para entender dónde se consume el tiempo de procesamiento, se realiza un análisis del registro de eventos (system.log). La función analizar_log parsea este archivo, identifica cada "Tarea completada", extrae su duración individual y agrupa los resultados por tipo de tarea (integral, derivada, tesis). Con estos datos se calculan los promedios y totales para la Tabla 4.

5. Resultados

5.1. Resultados Experimentales

En esta sección se presentan los datos cuantitativos recopilados durante la ejecución de las pruebas del sistema. Se busca mostrar objetivamente el rendimiento y la integridad de los datos bajo diferentes configuraciones.

Tabla 1: Comparativa de rendimiento: Secuencial vs. Concurrente.

Modo de Ejecución	Hilos (Semáforo)	Tiempo Total (s)	Speedup
Secuencial	1	0.2558	1.00x
Concurrente	5	0.0811	3.15x
Concurrente	10	0.0742	3.45x
Concurrente	20	0.0797	3.21x
Concurrente	50	0.0751	3.41x

La Tabla 1 registra el tiempo total de ejecución para cada modo. Se observa el tiempo base de la ejecución secuencial (0.2558 s) y los tiempos obtenidos al introducir concurrencia, junto con el Speedup (aceleración) calculado en relación con el tiempo base.

Tabla 2: Impacto del nivel de concurrencia (valor del semáforo).

Hilos Permitidos (N)	Tiempo Total (s)	Speedup (vs. N=1)
1	0.2558	1.00x
5	0.0811	3.15x
10	0.0742	3.45x
20	0.0797	3.21x
50	0.0751	3.41x

La Tabla 2 presenta los mismos datos de rendimiento, pero enfocándose en cómo la variación del valor del semáforo (N) impacta el tiempo total y el Speedup resultante.

Tabla 3: Impacto y costo de overhead de los *Locks* (con N=10).

Configuración	Tiempo Total (s)	¿Resultados Corruptos?
Con <i>Locks</i> (Implementación final)	0.0742	No
Sin <i>Locks</i> (Experimental)	0.0826	Sí

La Tabla 3 documenta el tiempo total la implementación final (con locks) contra una versión experimental sin ellos (con un nivel de concurrencia N=10) y, crucialmente, registra si los resultados generados se mantuvieron correctos ('No') o si presentaron corrupción de datos ('Sí') en ausencia de los locks.

Tabla 4: Análisis del tiempo de procesamiento por tipo de tarea.

Tipo de Tarea	Cantidad Total	T. Promedio (s)	T. Total (s)
Integral	42	0.023153	0.9724
Derivada	24	0.009418	0.2260
Tesis	42	0.017257	0.7248

La Tabla 4 detalla la cantidad total de tareas procesadas por tipo, el tiempo promedio que tomó cada una y el tiempo total acumulado por categoría.

Tabla 5: Verificación de integridad de datos para `largo.txt`.

Valor Esperado (Correcto): 58

Ejecución Concurrente	Valor Obtenido	¿Resultado Correcto?
N=1 (Secuencial)	58	Sí
N=5	58	Sí
N=10	58	Sí
N=20	58	Sí
N=50	58	Sí
N=10 (Sin <i>Lock</i>)	43	No

La Tabla 5 contrasta el valor esperado (58) con el valor obtenido en las diferentes ejecuciones. Se documenta que todas las ejecuciones concurrentes con locks (N=5 a N=50) produjeron el resultado correcto, mientras que la ejecución experimental (N=10 Sin Lock) arrojó un valor incorrecto (43).

5.2. Registros y Archivos de Salida

Los datos completos generados durante las pruebas se encuentran en los Apéndices.

- El Apéndice C (PRUEBA DEL SISTEMA) contiene el resumen comparativo de valores impreso en la consola al finalizar la ejecución de todas las pruebas.
- El Apéndice D (ARCHIVOS DE SALIDA) muestra el contenido final de los archivos generados: `integrales.txt` (D.2), `tesis.txt` (D.3) y `largo.txt` (D.4).
- El Apéndice E (REGISTRO DE EJECUCIÓN (LOG)) proporciona el log detallado de la ejecución.

Es importante aclarar que el archivo `largo.txt` (Apéndice D.4) muestra el valor 'corrupto' (43). Esto se debe a que la prueba experimental 'Sin Locks' (documentada en la Tabla 5 como 'INCORRECTO') fue la última prueba que se ejecutó. Por diseño, su resultado sobrescribió el valor correcto (58) que había sido generado por las pruebas estándar anteriores.

6. Conclusiones

En este laboratorio se logro implementar y evaluar un sistema concurrente en Python para el envío y procesamiento de tareas heterogéneas (integrales, derivadas y conteo de palabras). Las mediciones experimentales; registradas en la Sección 5 y detalladas en el Apéndice C, D y E, nos permiten llegar a las siguientes conclusiones.

- **Mejora de rendimiento mediante concurrencia:** La ejecución concurrente reduce de forma significativa el tiempo total frente al modo secuencial. El uso de semáforos para limitar el número de hilos activos produjo un *speedup* notable (véase la Tabla 1 y la Tabla 2). La ganancia máxima observada se alcanzó en los niveles intermedios de concurrencia (valor de semáforo ≈ 10), más allá de los cuales el beneficio adicional se atribuye al overhead de gestión de hilos y de sincronización.
- **Necesidad de sincronización para preservar la integridad:** Las pruebas sin locks mostraron resultados inconsistentes en el fichero compartido `largo.txt` (valor distinto al esperado), lo que confirma la aparición de condiciones de carrera. El coste en tiempo de introducir locks es pequeño en el escenario evaluado y justifica su uso cuando la valides de los datos compartidos es requerida (ver Tabla 3).
- **Comportamiento según tipo de tarea:** El análisis por tipo mostró que las tareas de *integral* presentan mayor tiempo medio de ejecución, seguidas por las de tipo *tesis* y, por último, las *derivadas* (Tabla 4). Esto es consistente con la carga computacional implícita en operaciones simbólicas frente a operaciones de conteo o evaluaciones simbólicas más simples.
- **Escalabilidad práctica y límites:** El paralelismo entrega beneficios claros hasta un rango moderado de hilos; sin embargo, existe un punto de retorno decreciente donde la coordinación, la contención por recursos (I/O, locks) y el overhead de los hilos impiden mejorar la latencia global. Para cargas mixtas CPU/E/S (como las del ejercicio), es necesario evaluar caso por caso el valor óptimo de semáforo en la máquina objetivo.

Recomendaciones prácticas

1. Para escenarios donde la *valides de los datos* es prioridad, mantener la estrategia actual basada en locks por archivo (o migrar a estructuras transaccionales) ya que su impacto temporal es reducido comparado con el riesgo de corrupción de datos.
2. Para maximizar throughput en sistemas productivos, calibrar empíricamente el parámetro del semáforo (N) en la máquina de despliegue usando la misma carga de trabajo; en este experimento los valores intermedios (ej. N=10) proporcionaron el mejor balance entre paralelismo y overhead.
3. Si el objetivo fuera acelerar tareas puramente CPU-bound a gran escala, considerar alternativas a hilos en CPython (p. ej. `multiprocessing` o migración a un entorno sin GIL) para explotar múltiples núcleos físicos de forma más efectiva.
4. Mantener un registro detallado (logging) y pruebas con y sin sincronización como parte del protocolo experimental para detectar condiciones de carrera y estimar el coste real de los mecanismos de exclusión.

Estos resultados permiten afirmar que se han cumplido los objetivos planteados: la implementación concurrente con hilos, el control mediante semáforos, la sincronización con locks y el envío de tareas desde múltiples clientes fueron evaluados experimentalmente y mostraron comportamientos coherentes con lo esperado.

En conclusión, el sistema implementado demuestra de forma empírica que la concurrencia bien gestionada ofrece mejoras de rendimiento relevantes y que los mecanismos de sincronización son imprescindibles para garantizar la integridad de los resultados compartidos; la selección del grado de concurrencia debe hacerse evaluando el trade-off entre *speedup* y overhead en la plataforma objetivo.

Referencias

- [1] Downey, A. B. (2012). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
- [2] Morero, F., *Introducción a la OOP*, versión 1.0.0, Montevideo, Grupo EIDOS, 2000.
- [3] Python Software Foundation. (2024). *Global Interpreter Lock*. Python Developer's Guide. Recuperado de <https://docs.python.org/3/glossary.html#term-global-interpreter-lock>
- [4] Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python Documentation. Recuperado de <https://docs.python.org/3/library/threading.html>
- [5] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- [6] SymPy Developers. (2024). *SymPy Documentation*. Recuperado de <https://docs.sympy.org/>

A. CÓDIGO-FUENTE EN LENGUAJE PYTHON

```
1 # Carga bibliotecas necesarias
2 import threading          # Para crear y manejar hilos
3 import time               # Para medir tiempos de ejecucion
4 import logging            # Para registro de eventos y errores
5 import sympy              # Para calculo simbolico
6 from sympy.parsing.sympy_parser import parse_expr # Convierte texto en
    formulas de sympy
7 import os                 # Para interactuar con el sistema (archivos,
    carpetas)
8 import traceback          # Para marcar errores detallados en logs
9 import re                 # Para expresiones regulares
10 from collections import defaultdict # Un diccionario con valores por defecto
11
12 # constantes globales
13 VALORES_N_SEMAFORO = [1, 5, 10, 20, 50] # Lista de hilos para las pruebas
14 N_PARA_PRUEBA_LOCK = 10                 # Valor fijo para pruebas de locks
15 DEMANDA_FILE = "demanda.txt"            # Nombre del archivo de tareas
16 LOG_FILE = "system.log"                  # Nombre del archivo de registro
17
18 # CLASE BaseDeDatos. Molde para gestionar la concurrencia; clase que
    representa la 'base de datos' del servidor central
19 class BaseDeDatos:
20     # Constructor, se ejecuta al crear la clase
21     def __init__(self, max_hilos_concurrentes):
22         # Semaforo que limita los hilos concurrentes, controla cuantos hilos
            pueden ejecutar a la vez
23         self.semaforo = threading.Semaphore(max_hilos_concurrentes)
24         # Locks para proteger el acceso a los archivos; aseguran que solo un
            hilo escriba a la vez
25         self.lock_integrales = threading.Lock()    # Candado para 'integrales
            .txt'
26         self.lock_derivadas = threading.Lock()    # Candado para 'derivadas.
            txt'
27         self.lock_tesis = threading.Lock()        # Candado para 'tesis.txt'
28         self.lock_largo = threading.Lock()        # Candado para 'largo.txt'
29
30 # Esta es la funcion que se ejecutara en cada hilo
31 def procesar_tarea(tarea, bd, use_locks=True):
32     thread_name = threading.current_thread().name # Obtiene el nombre del
        hilo actual (ej "Thread-1") para usarlo en los logs
33
34     # seccion controlada por Semaforo
35     # 'with' maneja el 'acquire' y 'release' del semaforo automaticamente,
        el hilo pide "un lugar" en el semaforo. Si esta lleno, espera aqui
36     with bd.semaforo:
37         # Se usa un 'try' grande para asegurar que el hilo no muera por un
            error
38         try:
39             logging.info(f"[{thread_name}] Inicia procesamiento de tarea ID
                {tarea['id']} ({tarea['tipo']})") # Registra en el log que
                esta tarea (y este hilo) han comenzado
40             task_start_time = time.time() # Inicia el cronometro para medir
                cuanto tarda esta tarea
41
42             # Sacar los datos de la tarea
43             tipo = tarea['tipo']
44             datos = tarea['datos']
```

```

45     resultado_procesado = None # Prepara una variable para el
        resultado
46
47     # logica de procesamiento
48     # Si la tarea es una 'integral'
49     if tipo == 'integral':
50         x = sympy.symbols('x') # Define 'x' como variable simbolica
51         try:
52             expr = parse_expr(datos) # Convierte el texto de 'datos'
                en una formula
53             resultado_procesado = str(sympy.integrate(expr, x)) #
                Calcula la integral
54         except Exception as e:
55             resultado_procesado = f"Error al integrar: {e}" #
                Captura errores de sympy
56             escribir_en_archivo("integrales.txt", f"Integral de '{datos
                }': {resultado_procesado}\n", bd.lock_integrales,
                use_locks) # Llama a la funcion para escribir, pasandole
                el candado especifico de integrales
57
58     # Si la tarea es una 'derivada'
59     elif tipo == 'derivada':
60         x = sympy.symbols('x') # Define 'x' como variable simbolica
61         try:
62             expr = parse_expr(datos) # Convierte el texto en formula
63             resultado_procesado = str(sympy.diff(expr, x)) # Calcula
                la derivada
64         except Exception as e:
65             resultado_procesado = f"Error al derivar: {e}" # Captura
                errores de sympy
66             escribir_en_archivo("derivadas.txt", f"Derivada de '{datos
                }': {resultado_procesado}\n", bd.lock_derivadas, use_locks
                ) # Llama a escribir, pero esta vez usando el candado de
                derivadas
67
68     # Si la tarea es una 'tesis' (conteo de palabras)
69     elif tipo == 'tesis':
70         palabras = datos.split() # Separa el texto por espacios
71         num_palabras = len(palabras) # Cuenta la cantidad de
                palabras
72         resultado_procesado = f"Tesis (ID: {tarea['id']}) - {
                num_palabras} palabras"
73         escribir_en_archivo("tesis.txt", f"{resultado_procesado}\n",
                bd.lock_tesis, use_locks) # Escribe el resultado en '
                tesis.txt' (con su candado)
74         actualizar_conteo_largo(num_palabras, bd.lock_largo,
                use_locks) # Llama a otra funcion para actualizar el
                conteo en 'largo.txt'
75     # fin de procesamiento
76     task_end_time = time.time() # Detiene el cronometro
77     duracion = task_end_time - task_start_time # Calcula cuanto
                tarda
78     logging.info(f"[{thread_name}] Tarea completada; TIPO: {tipo};
                DURACION: {duracion:.6f}s") # Registra en el log que la tarea
                se completo y cuanto demoro
79     except Exception as e:
80         logging.error(f"[{thread_name}] Error inesperado: {e}") #
                Captura cualquier error inesperado que no haya sido capturado
                adentro
81     # Al salir del 'with', el hilo automaticamente libera su lugar en el
        semaforo permitiendo que otro hilo que esperaba pueda entrar

```

```

82
83 # Funcion de ayuda para escribir contenido en un archivo
84 def escribir_en_archivo(filepath, contenido, lock, use_locks):
85     # Revisa si la ejecucion actual debe usar candados (para pruebas)
86     if use_locks:
87         # 'with lock' es la forma segura de adquirir y liberar el candado,
            el hilo espera aqui si el candado esta ocupado por otro hilo
88         with lock:
89             # Una vez tiene el candado, abre el archivo
90             with open(filepath, "a", encoding="utf-8") as f:
91                 f.write(contenido)
92     else:
93         # Rama para pruebas sin candados
94         with open(filepath, "a", encoding="utf-8") as f:
95             f.write(contenido)
96
97 # Funcion para 'largo.txt' porque necesita leer y luego escribir
98 def actualizar_conteo_largo(num_palabras, lock, use_locks):
99     if use_locks:
100         # Adquiere el candado especifico para 'largo.txt'
101         with lock:
102             total_actual = 0 # Valor por defecto
103             try:
104                 # leer el valor actual
105                 with open("largo.txt", "r", encoding="utf-8") as f:
106                     total_actual = int(f.read().strip() or 0) # Lee el
                        contenido, 'strip' quita espacios/saltos, 'or 0'
                        previene error si el archivo esta vacio
107             except FileNotFoundError:
108                 pass # Si el archivo no existe, 'total_actual' sigue en 0
109             nuevo_total = total_actual + num_palabras # modifica el valor
110             # escribe el nuevo valor
111             with open("largo.txt", "w", encoding="utf-8") as f:
112                 f.write(str(nuevo_total))
113     else:
114         # La misma logica pero sin candado
115         total_actual = 0
116         try:
117             with open("largo.txt", "r", encoding="utf-8") as f:
118                 total_actual = int(f.read().strip() or 0)
119         except FileNotFoundError:
120             pass
121         nuevo_total = total_actual + num_palabras
122         with open("largo.txt", "w", encoding="utf-8") as f:
123             f.write(str(nuevo_total))
124
125 # Esta funcion recibe la lista de tareas y la 'bd' (con los locks/semaforo)
126 def procesar_tareas(lista_tareas, bd, use_locks=True):
127     hilos = [] # Lista para guardar los hilos que creemos
128     logging.info(f"Iniciando procesamiento de {len(lista_tareas)} tareas..."
129         ) # Registra en el log cuantas tareas se van a procesar
130     # Recorre la lista de tareas
131     for tarea in lista_tareas:
132         # Por cada tarea, crea un objeto Hilo (Thread)
133         t = threading.Thread(
134             target=procesar_tarea, # 'target' es la funcion que debe
                ejecutar el hilo
135             args=(tarea, bd, use_locks), # Argumentos que se le pasan a '
                procesar_tarea'
136             name=f"Hilo-ID-{tarea['id']}" # Le da un nombre al hilo para el
                log

```

```

136         )
137         hilos.append(t) # agrega el hilo recién creado a la lista
138         t.start() # Inicia el hilo (comienza a ejecutar 'procesar_tarea')
139
140         logging.info(f"Se lanzaron {len(hilos)} hilos. Esperando finalización (
141             join)...") # Informa que ya se lanzaron todos los hilos
142
143         # Segundo bucle, esta vez para esperar a que terminen
144         for t in hilos:
145             t.join() # bloquea el programa principal aquí hasta que este hilo '
146                 t' haya terminado su ejecución
147         logging.info("--- Todas las tareas han sido procesadas. ---") # Mensaje
148             final cuando todos se completaron
149
150     # Función para configurar el sistema de logging
151     def setup_logging():
152         # Configura el log principal (el que escribe al archivo)
153         logging.basicConfig(
154             level=logging.INFO, # Nivel mínimo de registro
155             format='%(asctime)s - %(threadName)s - %(message)s', # Formato del
156                 log: [Tiempo] - [NombreDelHilo] - [Mensaje]
157             filename=LOG_FILE, # Nombre del archivo
158             filemode='w' # sobrescribe el log cada vez
159         )
160         # Configuración para ver el log en la consola
161         console = logging.StreamHandler() # Crea un manejador para la consola
162         console.setLevel(logging.INFO) # Nivel mínimo para la consola
163         formatter = logging.Formatter('%(asctime)s - %(message)s') # Un formato
164             más simple para la consola
165         console.setFormatter(formatter) # Aplica ese formato
166         logging.getLogger('').addHandler(console) # agrega el manejador de
167             consola al logger raíz así los logs se escriben en el archivo y en la
168             consola a la vez
169
170     # Función para crear 'demanda.txt' si no se encuentra
171     def crear_demanda_si_no_existe(filepath):
172         # Revisa si el archivo no existe en el disco
173         if not os.path.exists(filepath):
174             logging.info(f"Creando archivo '{filepath}' con datos de ejemplo...")
175             # Si no existe, informa en el log que va a crearlo
176             # Define el contenido de ejemplo que se escribirá
177             contenido = """integral:x**2
178 derivada:sin(x)
179 tesis>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
180 integral:cos(x)*x
181 integral:exp(x**2)
182 tesis:Etiam interdum massa sit amet nibh scelerisque euismod.
183 derivada:x**5 + 2*x
184 integral:1/x
185 tesis:Integer in arcu in elit sagittis luctus.
186 tesis:Phasellus et dui sed metus condimentum eleifend id ac erat.
187 derivada:tan(x)
188 integral:x**3
189 integral:sin(x)**2
190 tesis:Cras cursus leo ut diam hendrerit, id ornare nulla iaculis.
191 derivada:log(x)
192 tesis:Praesent sed nulla maximus, scelerisque magna in, commodo ante.
193 integral:x*cos(x)
194 tesis:Duis eleifend tellus eu imperdiet pretium.
195 """
196             # Intenta escribir el archivo

```

```

189         try:
190             # Abre el archivo en modo write; sobrescribe
191             with open(filepath, "w", encoding="utf-8") as f:
192                 f.write(contenido) # Escribe el contenido de ejemplo
193         except IOError as e:
194             logging.error(f"No se pudo crear el archivo de demanda: {e}") #
195                 Si falla la escritura, lo registra en el log de error
196
197 # Lee el archivo 'demanda.txt' y lo convierte en una lista de diccionarios
198 def leer_tareas_demanda(filepath):
199     tareas = [] # Lista para guardar las tareas
200     try:
201         # Abre el archivo en modo leer
202         with open(filepath, "r", encoding="utf-8") as f:
203             # Lee el archivo linea por linea, 'i' es el numero de linea
204             for i, linea in enumerate(f):
205                 # Si la linea no tiene ':', la ignora (linea vacia o
206                 # malformada)
207                 if ":" not in linea:
208                     continue # Salta a la siguiente linea
209
210                 partes = linea.strip().split(":", 1) # Limpia espacios y
211                 # divide por el primer ':'
212                 # Si la division fue exitosa (dio 2 partes)
213                 if len(partes) == 2:
214                     # agrega la tarea como un diccionario a la lista
215                     tareas.append({
216                         "id": i, # Usa el numero de linea como ID
217                         "tipo": partes[0].strip(), # La parte 0 es el tipo
218                         "datos": partes[1].strip() # La parte 1 son los
219                             datos
220                     })
221     except FileNotFoundError:
222         logging.error(f"Error: No se encontro el archivo '{filepath}'.") #
223             Maneja el error si el archivo de demanda no existe
224         return [] # Devuelve una lista vacia
225     logging.info(f"Leidas {len(tareas)} tareas desde '{filepath}'.") #
226         Informa en el log cuantas tareas se cargaron
227     return tareas # Devuelve la lista de tareas
228
229 # Borra los archivos de resultados antes de empezar una nueva ejecucion
230 def preparar_archivos_salida():
231     archivos = ["integrales.txt", "derivadas.txt", "tesis.txt"]
232     for f in archivos:
233         try:
234             open(f, "w").close() # Abre en modo write y cierra; vacia el
235                 archivo
236         except IOError as e:
237             logging.error(f"Error al limpiar {f}: {e}") # Captura si hay
238                 error de permisos
239
240 # Para 'largo.txt', lo inicializa en 0
241 try:
242     with open("largo.txt", "w") as f:
243         f.write("0")
244 except IOError as e:
245     logging.error(f"Error al inicializar largo.txt: {e}")
246
247 # Funcion de ayuda para leer el resultado final de 'largo.txt'
248 def leer_valor_largo():
249     try:
250         with open("largo.txt", "r") as f:

```

```

242         valor = f.read().strip() # Lee el contenido
243         return int(valor or 0) # Convierte a numero (o 0 si estaba vacio
    )
244     except (FileNotFoundError, ValueError):
245         # Si no existe o el contenido no es un numero
246         return -1 # Devuelve -1 para indicar error
247
248 # Analiza el archivo de log para extraer metricas de tiempo
249 def analizar_log(log_file=LOG_FILE):
250     tiempos = defaultdict(list) # Un diccionario que crea listas vacias para
        tipos nuevos
251     regex = r"Tarea completada; TIPO: (\w+); DURACION: (\d+\.\d+)s" #
        Expresion regular para buscar las lineas de 'Tarea completada',
        captura 2 grupos: (el tipo) y (la duracion)
252     try:
253         # Abre el archivo de log en modo lectura, evita fallos por
        caracteres invalidos
254         with open(log_file, "r", encoding="utf-8", errors="replace") as f:
255             # Lee el log linea por linea
256             for linea in f:
257                 match = re.search(regex, linea) # Busca el patron regex en
                    la linea actual
258                 # Si encontro el patron
259                 if match:
260                     tipo_tarea = match.group(1) # Grupo 1 es el tipo (
                        integral, derivada, etc)
261                     duracion = float(match.group(2)) # Grupo 2 es la
                        duracion
262                     tiempos[tipo_tarea].append(duracion) # Agrega la
                        duracion a la lista de ese tipo
263             # Si el diccionario 'tiempos' esta vacio (no encontro nada)
264             if not tiempos:
265                 logging.warning(f"No se encontraron datos de TAREA en '{log_file
                    }'".)
266                 return # Termina la funcion
267             print("\n" + "=" * 60)
268             print(f"    ANALISIS DE TIEMPOS DE EJECUCION POR TAREA")
269             print(f"    (Datos extraidos de '{log_file}')

```

```

286 setup_logging() # Configura el logging (para que escriba en archivo y
      consola)
287 crear_demanda_si_no_existe(DEMANDA_FILE) # Asegura que el archivo '
      demanda.txt' exista
288
289 # secuencial (Base de comparacion)
290 logging.info("--- INICIO EXPERIMENTO BASE (N=1, Secuencial) ---")
291 preparar_archivos_salida() # Limpia los .txt de resultados anteriores
292 lista_tareas = leer_tareas_demanda(DEMANDA_FILE) # Lee las tareas a
      realizar
293 if not lista_tareas:
294     logging.error("No se pudieron leer las tareas.")
295     return
296 bd_secuencial = BaseDeDatos(max_hilos_concurrentes=1) # Crea la BD con
      semaforo de 1 (solo 1 hilo a la vez)
297 start_time = time.time() # Inicia cronometro
298 procesar_tareas(lista_tareas, bd_secuencial, use_locks=True) # Procesa
      todas las tareas, una por una
299 end_time = time.time() # Detiene cronometro
300
301 tiempo_secuencial = end_time - start_time # Tiempo de referencia (el mas
      lento)
302 valor_esperado_largo = leer_valor_largo() # Lee el resultado de 'largo.
      txt'
303
304 logging.info(f"--- FIN EXPERIMENTO BASE (N=1) ---")
305 logging.info(f"Tiempo Total: {tiempo_secuencial:.4f}s")
306 logging.info(f"Valor 'largo.txt' (Esperado): {valor_esperado_largo}\n")
307
308 # concurrente (con Locks)
309 # Diccionario para guardar los resultados (tiempo y valor) de cada
      prueba
310 resultados_concurrencia = {}
311
312 # Itera sobre la lista de hilos a probar
313 for n in VALORES_N_SEMAFORO:
314     if n == 1:
315         resultados_concurrencia[1] = {'tiempo': tiempo_secuencial, '
            largo': valor_esperado_largo} # Si N=1, ya se hizo, solo
            guardamos el resultado
316         continue # Salta al siguiente valor de N
317
318     # Prueba para N > 1
319     logging.info(f"--- INICIO EXPERIMENTO (N={n}, Con Locks) ---")
320     preparar_archivos_salida() # Limpia los archivos
321     bd = BaseDeDatos(max_hilos_concurrentes=n) # Crea la BD con un
            semaforo de N hilos
322     start_time = time.time() # Inicia cronometro
323     procesar_tareas(lista_tareas, bd, use_locks=True) # Procesa todo con
            N hilos y con locks
324     end_time = time.time() # Detiene cronometro
325
326     tiempo_total = end_time - start_time
327     valor_largo = leer_valor_largo() # Lee el resultado de 'largo.txt'
328     resultados_concurrencia[n] = {'tiempo': tiempo_total, 'largo':
            valor_largo} # Guarda el resultado para este N
329
330     logging.info(f"--- FIN EXPERIMENTO (N={n}) ---")
331     logging.info(f"Tiempo Total: {tiempo_total:.4f}s")
332     logging.info(f"Valor 'largo.txt' (Obtenido): {valor_largo}\n")
333

```

```

334 # concurrente (sin Locks)
335 logging.info(f"--- INICIO EXPERIMENTO (N={N_PARA_PRUEBA_LOCK}, SIN Locks
    ) ---")
336 preparar_archivos_salida() # Limpia archivos
337 bd_sin_lock = BaseDeDatos(max_hilos_concurrentes=N_PARA_PRUEBA_LOCK)
338 start_time_sin_lock = time.time()
339 procesar_tareas(lista_tareas, bd_sin_lock, use_locks=False) # Ejecuta el
    procesamiento pero con use_locks=False
340 end_time_sin_lock = time.time()
341 tiempo_sin_lock = end_time_sin_lock - start_time_sin_lock
342 valor_largo_sin_lock = leer_valor_largo() # Lee el valor de 'largo.txt'
343
344 logging.info(f"--- FIN EXPERIMENTO (SIN Locks) ---")
345 logging.info(f"Tiempo Total: {tiempo_sin_lock:.4f}s")
346 logging.info(f"Valor 'largo.txt' (Obtenido): {valor_largo_sin_lock} (
    Probablemente incorrecto)\n")
347
348 # impresion de reportes finales en consola
349 print("\n\n" + "=" * 60)
350 print("          RESUMEN DE EXPERIMENTOS DE RENDIMIENTO")
351 print("=" * 60 + "\n")
352
353 # Reporte 1: Speedup
354 print("Análisis de Concurrencia (Efecto del Semaforo)")
355 print("Comparativa de tiempo total vs. ejecucion secuencial (N=1).\n")
356 # Itera sobre los resultados guardados
357 for n, res in resultados_concurrencia.items():
358     if res['tiempo'] > 0:
359         speedup = tiempo_secuencial / res['tiempo'] # Speedup = Tiempo
            (1) / Tiempo(N)
360     else:
361         speedup = float('inf') # Evita division entre cero
362         etiqueta = "Secuencial" if n == 1 else "Concurrente"
363         print(f"    {etiqueta} (N={n}): \t{res['tiempo']:.4f}s (Speedup: {
            speedup:.2f}x)")
364
365 # Reporte 2: Costo de los Locks
366 print("\n" + "-" * 60)
367 print("Análisis de Sincronizacion (Efecto de Locks)")
368 print(f"Comparativa de rendimiento con/sin locks (usando N={
    N_PARA_PRUEBA_LOCK}).\n")
369 if N_PARA_PRUEBA_LOCK in resultados_concurrencia:
370     tiempo_con_lock = resultados_concurrencia[N_PARA_PRUEBA_LOCK]['
        tiempo']
371     print(f"    Con Locks: \t\t{tiempo_con_lock:.4f}s")
372 else:
373     print(f"    Con Locks: \t\tN/A (N={N_PARA_PRUEBA_LOCK} no se probó)")
374
375 print(f"    Sin Locks: \t\t{tiempo_sin_lock:.4f}s")
376
377 # Reporte 3: Integridad de Datos
378 print("\n" + "-" * 60)
379 print("Verificacion de Integridad de Datos (largo.txt)")
380 print(f"Comprobacion del valor final de conteo de palabras.\n")
381 print(f"    Valor Esperado (Correcto): {valor_esperado_largo}\n") #
    Muestra el valor que deberia dar
382 # Revisa las corridas con locks
383 for n, res in resultados_concurrencia.items():
384     es_correcto = (res['largo'] == valor_esperado_largo) # Compara el
        valor obtenido vs el esperado
385     resultado_str = "CORRECTO" if es_correcto else "INCORRECTO"

```

```

386         print(f"   Corrida N={n} (con locks): \t{res['largo']} -> {
              resultado_str}")
387     es_correcto_sin_lock = (valor_largo_sin_lock == valor_esperado_largo) #
        Revisa la corrida sin locks
388     resultado_str_sin_lock = "CORRECTO" if es_correcto_sin_lock else "
        INCORRECTO"
389     print(f"   Corrida N={N_PARA_PRUEBA_LOCK} (sin locks):\t{
        valor_largo_sin_lock} -> {resultado_str_sin_lock}")
390
391     # Llama a la funcion que analiza el log (system.log)
392     analizar_log()
393
394     print("\n" + "=" * 60)
395     print("Todos los experimentos y analisis han finalizado")
396     print("=" * 60)
397
398     # El script empieza a ejecutarse aqui
399     if __name__ == "__main__":
400         main() # Llama a la funcion principal

```

Fuente: autoria propia.

B. ARCHIVO DE ENTRADA

B.1. demanda.txt

```
integral:x**2
derivada:sin(x)
tesis>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
integral:cos(x)*x
integral:exp(x**2)
tesis:Etiam interdum massa sit amet nibh scelerisque euismod.
derivada:x**5 + 2*x
integral:1/x
tesis:Integer in arcu in elit sagittis luctus.
tesis:Phasellus et dui sed metus condimentum eleifend id ac erat.
derivada:tan(x)
integral:x**3
integral:sin(x)**2
tesis:Cras cursus leo ut diam hendrerit, id ornare nulla iaculis.
derivada:log(x)
tesis:Praesent sed nulla maximus, scelerisque magna in, commodo ante.
integral:x*cos(x)
tesis:Duis eleifend tellus eu imperdiet pretium.
```

Fuente: autoria propia.

C. PRUEBA DEL SISTEMA

RESUMEN DE EXPERIMENTOS DE RENDIMIENTO

Analisis de Concurrencia (Efecto del Semaforo)
Comparativa de tiempo total vs. ejecucion secuencial (N=1).

```
Secuencial (N=1): 0.2558s (Speedup: 1.00x)
Concurrente (N=5): 0.0811s (Speedup: 3.15x)
Concurrente (N=10): 0.0742s (Speedup: 3.45x)
Concurrente (N=20): 0.0797s (Speedup: 3.21x)
Concurrente (N=50): 0.0751s (Speedup: 3.41x)
```

Analisis de Sincronizacion (Efecto de Locks)
Comparativa de rendimiento con/sin locks (usando N=10).

```
Con Locks: 0.0742s
Sin Locks: 0.0826s
```

Verificacion de Integridad de Datos (largo.txt)
Comprobacion del valor final de conteo de palabras.

Valor Esperado (Correcto): 58

```
Corrida N=1 (con locks): 58 -> CORRECTO
Corrida N=5 (con locks): 58 -> CORRECTO
Corrida N=10 (con locks): 58 -> CORRECTO
Corrida N=20 (con locks): 58 -> CORRECTO
Corrida N=50 (con locks): 58 -> CORRECTO
```

Corrida N=10 (sin locks): 43 -> INCORRECTO

```
=====
ANALISIS DE TIEMPOS DE EJECUCION POR TAREA
(Datos extraidos de 'system.log')
=====
```

Tipo Tarea	Cantidad	T. Promedio (s)	T. Total (s)
Integral	42	0.023153	0.9724
Derivada	24	0.009418	0.2260
Tesis	42	0.017257	0.7248

```
=====
Todos los experimentos y analisis han finalizado
=====
```

Fuente: autoria propia.

D. ARCHIVOS DE SALIDA

D.1. derivadas.txt

```
Derivada de 'sin(x)': cos(x)
Derivada de 'x**5 + 2*x': 5*x**4 + 2
Derivada de 'tan(x)': tan(x)**2 + 1
Derivada de 'log(x)': 1/x
```

Fuente: autoria propia.

D.2. integrales.txt

```
Integral de 'x**2': x**3/3
Integral de 'cos(x)*x': x*sin(x) + cos(x)
Integral de 'exp(x**2)': sqrt(pi)*erfi(x)/2
Integral de '1/x': log(x)
Integral de 'x**3': x**4/4
Integral de 'sin(x)**2': x/2 - sin(x)*cos(x)/2
Integral de 'x*cos(x)': x*sin(x) + cos(x)
```

Fuente: autoria propia.

D.3. tesis.txt

```
Tesis (ID: 2) - 8 palabras
Tesis (ID: 5) - 8 palabras
Tesis (ID: 8) - 7 palabras
Tesis (ID: 9) - 10 palabras
Tesis (ID: 13) - 10 palabras
Tesis (ID: 15) - 9 palabras
Tesis (ID: 17) - 6 palabras
```

Fuente: autoria propia.

D.4. largo.txt

43

Fuente: autoria propia.

E. REGISTRO DE EJECUCIÓN (LOG)

2025-11-11 12:29:04,338 - Creando archivo 'demanda.txt'
con datos de ejemplo...
2025-11-11 12:29:04,339 - --- INICIO EXPERIMENTO BASE (N=1,
Secuencial) ---
2025-11-11 12:29:04,349 - Leds 18 tareas desde 'demanda.txt'.
2025-11-11 12:29:04,349 - Iniciando procesamiento de 18 tareas...
2025-11-11 12:29:04,350 - [Hilo-ID-0] Inicia procesamiento de
tarea ID 0 (integral)
2025-11-11 12:29:04,358 - Se lanzaron 18 hilos. Esperando
finalizacin (join)...
2025-11-11 12:29:04,419 - [Hilo-ID-0] Tarea completada; TIPO:
integral; DURACION: 0.068443s
2025-11-11 12:29:04,420 - [Hilo-ID-1] Inicia procesamiento de
tarea ID 1 (derivada)
2025-11-11 12:29:04,433 - [Hilo-ID-1] Tarea completada; TIPO:
derivada; DURACION: 0.004475s
2025-11-11 12:29:04,434 - [Hilo-ID-2] Inicia procesamiento de
tarea ID 2 (tesis)
2025-11-11 12:29:04,434 - [Hilo-ID-2] Tarea completada; TIPO:
tesis; DURACION: 0.000779s
2025-11-11 12:29:04,434 - [Hilo-ID-3] Inicia procesamiento de
tarea ID 3 (integral)
2025-11-11 12:29:04,497 - [Hilo-ID-3] Tarea completada; TIPO:
integral; DURACION: 0.062867s
2025-11-11 12:29:04,497 - [Hilo-ID-4] Inicia procesamiento de
tarea ID 4 (integral)
2025-11-11 12:29:04,566 - [Hilo-ID-4] Tarea completada; TIPO:
integral; DURACION: 0.068529s
2025-11-11 12:29:04,566 - [Hilo-ID-5] Inicia procesamiento de
tarea ID 5 (tesis)
2025-11-11 12:29:04,567 - [Hilo-ID-5] Tarea completada; TIPO:
tesis; DURACION: 0.000427s
2025-11-11 12:29:04,567 - [Hilo-ID-6] Inicia procesamiento de
tarea ID 6 (derivada)
2025-11-11 12:29:04,572 - [Hilo-ID-6] Tarea completada; TIPO:
derivada; DURACION: 0.004025s
2025-11-11 12:29:04,572 - [Hilo-ID-7] Inicia procesamiento de
tarea ID 7 (integral)
2025-11-11 12:29:04,575 - [Hilo-ID-7] Tarea completada; TIPO:
integral; DURACION: 0.002210s
2025-11-11 12:29:04,575 - [Hilo-ID-8] Inicia procesamiento de
tarea ID 8 (tesis)
2025-11-11 12:29:04,576 - [Hilo-ID-8] Tarea completada; TIPO:
tesis; DURACION: 0.000944s
2025-11-11 12:29:04,576 - [Hilo-ID-9] Inicia procesamiento de
tarea ID 9 (tesis)
2025-11-11 12:29:04,577 - [Hilo-ID-9] Tarea completada; TIPO:
tesis; DURACION: 0.000996s
2025-11-11 12:29:04,578 - [Hilo-ID-10] Inicia procesamiento de
tarea ID 10 (derivada)
2025-11-11 12:29:04,581 - [Hilo-ID-10] Tarea completada; TIPO:
derivada; DURACION: 0.003531s
2025-11-11 12:29:04,581 - [Hilo-ID-11] Inicia procesamiento de
tarea ID 11 (integral)
2025-11-11 12:29:04,584 - [Hilo-ID-11] Tarea completada; TIPO:
integral; DURACION: 0.002230s
2025-11-11 12:29:04,584 - [Hilo-ID-12] Inicia procesamiento de
tarea ID 12 (integral)

2025-11-11 12:29:04,587 - [Hilo-ID-12] Tarea completada; TIPO: integral; DURACION: 0.003765s
 2025-11-11 12:29:04,588 - [Hilo-ID-13] Inicia procesamiento de tarea ID 13 (tesis)
 2025-11-11 12:29:04,588 - [Hilo-ID-13] Tarea completada; TIPO: tesis; DURACION: 0.000444s
 2025-11-11 12:29:04,588 - [Hilo-ID-14] Inicia procesamiento de tarea ID 14 (derivada)
 2025-11-11 12:29:04,590 - [Hilo-ID-14] Tarea completada; TIPO: derivada; DURACION: 0.002027s
 2025-11-11 12:29:04,590 - [Hilo-ID-15] Inicia procesamiento de tarea ID 15 (tesis)
 2025-11-11 12:29:04,591 - [Hilo-ID-15] Tarea completada; TIPO: tesis; DURACION: 0.000358s
 2025-11-11 12:29:04,592 - [Hilo-ID-16] Inicia procesamiento de tarea ID 16 (integral)
 2025-11-11 12:29:04,603 - [Hilo-ID-16] Tarea completada; TIPO: integral; DURACION: 0.011326s
 2025-11-11 12:29:04,604 - [Hilo-ID-17] Inicia procesamiento de tarea ID 17 (tesis)
 2025-11-11 12:29:04,604 - [Hilo-ID-17] Tarea completada; TIPO: tesis; DURACION: 0.000000s
 2025-11-11 12:29:04,605 - --- Todas las tareas han sido procesadas. ---
 2025-11-11 12:29:04,605 - --- FIN EXPERIMENTO BASE (N=1) ---
 2025-11-11 12:29:04,605 - Tiempo Total: 0.2558s
 2025-11-11 12:29:04,605 - Valor 'largo.txt' (Esperado): 58

 2025-11-11 12:29:04,605 - --- INICIO EXPERIMENTO (N=5, Con Locks) ---
 2025-11-11 12:29:04,606 - Iniciando procesamiento de 18 tareas...
 2025-11-11 12:29:04,607 - [Hilo-ID-0] Inicia procesamiento de tarea ID 0 (integral)
 2025-11-11 12:29:04,608 - [Hilo-ID-1] Inicia procesamiento de tarea ID 1 (derivada)
 2025-11-11 12:29:04,609 - [Hilo-ID-2] Inicia procesamiento de tarea ID 2 (tesis)
 2025-11-11 12:29:04,609 - [Hilo-ID-0] Tarea completada; TIPO: integral; DURACION: 0.002310s
 2025-11-11 12:29:04,610 - [Hilo-ID-3] Inicia procesamiento de tarea ID 3 (integral)
 2025-11-11 12:29:04,610 - [Hilo-ID-1] Tarea completada; TIPO: derivada; DURACION: 0.001250s
 2025-11-11 12:29:04,610 - [Hilo-ID-4] Inicia procesamiento de tarea ID 4 (integral)
 2025-11-11 12:29:04,623 - [Hilo-ID-5] Inicia procesamiento de tarea ID 5 (tesis)
 2025-11-11 12:29:04,652 - [Hilo-ID-3] Tarea completada; TIPO: integral; DURACION: 0.041717s
 2025-11-11 12:29:04,652 - [Hilo-ID-6] Inicia procesamiento de tarea ID 6 (derivada)
 2025-11-11 12:29:04,653 - [Hilo-ID-7] Inicia procesamiento de tarea ID 7 (integral)
 2025-11-11 12:29:04,656 - [Hilo-ID-2] Tarea completada; TIPO: tesis; DURACION: 0.045438s
 2025-11-11 12:29:04,656 - [Hilo-ID-4] Tarea completada; TIPO: integral; DURACION: 0.032474s
 2025-11-11 12:29:04,656 - [Hilo-ID-8] Inicia procesamiento de tarea ID 8 (tesis)
 2025-11-11 12:29:04,656 - [Hilo-ID-6] Tarea completada; TIPO: derivada; DURACION: 0.003251s

2025-11-11 12:29:04,656 - [Hilo-ID-10] Inicia procesamiento de
 tarea ID 10 (derivada)
 2025-11-11 12:29:04,657 - [Hilo-ID-7] Tarea completada; TIPO:
 integral; DURACION: 0.002501s
 2025-11-11 12:29:04,657 - [Hilo-ID-12] Inicia procesamiento de
 tarea ID 12 (integral)
 2025-11-11 12:29:04,659 - [Hilo-ID-14] Inicia procesamiento de
 tarea ID 14 (derivada)
 2025-11-11 12:29:04,659 - [Hilo-ID-5] Tarea completada; TIPO:
 tesis; DURACION: 0.020727s
 2025-11-11 12:29:04,662 - [Hilo-ID-10] Tarea completada; TIPO:
 derivada; DURACION: 0.004549s
 2025-11-11 12:29:04,662 - [Hilo-ID-9] Inicia procesamiento de
 tarea ID 9 (tesis)
 2025-11-11 12:29:04,662 - [Hilo-ID-16] Inicia procesamiento de
 tarea ID 16 (integral)
 2025-11-11 12:29:04,673 - Se lanzaron 18 hilos. Esperando
 finalizacin (join)...
 2025-11-11 12:29:04,681 - [Hilo-ID-12] Tarea completada; TIPO:
 integral; DURACION: 0.022131s
 2025-11-11 12:29:04,681 - [Hilo-ID-14] Tarea completada; TIPO:
 derivada; DURACION: 0.019659s
 2025-11-11 12:29:04,681 - [Hilo-ID-11] Inicia procesamiento de
 tarea ID 11 (integral)
 2025-11-11 12:29:04,683 - [Hilo-ID-15] Inicia procesamiento de
 tarea ID 15 (tesis)
 2025-11-11 12:29:04,684 - [Hilo-ID-8] Tarea completada; TIPO:
 tesis; DURACION: 0.027279s
 2025-11-11 12:29:04,684 - [Hilo-ID-16] Tarea completada; TIPO:
 integral; DURACION: 0.020298s
 2025-11-11 12:29:04,684 - [Hilo-ID-13] Inicia procesamiento de
 tarea ID 13 (tesis)
 2025-11-11 12:29:04,684 - [Hilo-ID-17] Inicia procesamiento de
 tarea ID 17 (tesis)
 2025-11-11 12:29:04,684 - [Hilo-ID-11] Tarea completada; TIPO:
 integral; DURACION: 0.001993s
 2025-11-11 12:29:04,685 - [Hilo-ID-9] Tarea completada; TIPO:
 tesis; DURACION: 0.022938s
 2025-11-11 12:29:04,685 - [Hilo-ID-15] Tarea completada; TIPO:
 tesis; DURACION: 0.001993s
 2025-11-11 12:29:04,686 - [Hilo-ID-13] Tarea completada; TIPO:
 tesis; DURACION: 0.001994s
 2025-11-11 12:29:04,686 - [Hilo-ID-17] Tarea completada; TIPO:
 tesis; DURACION: 0.001994s
 2025-11-11 12:29:04,686 - --- Todas las tareas han sido
 procesadas. ---
 2025-11-11 12:29:04,687 - --- FIN EXPERIMENTO (N=5) ---
 2025-11-11 12:29:04,687 - Tiempo Total: 0.0811s
 2025-11-11 12:29:04,687 - Valor 'largo.txt' (Obtenido): 58

 2025-11-11 12:29:04,687 - --- INICIO EXPERIMENTO (N=10, Con
 Locks) ---
 2025-11-11 12:29:04,688 - Iniciando procesamiento de 18 tareas...
 2025-11-11 12:29:04,689 - [Hilo-ID-0] Inicia procesamiento de
 tarea ID 0 (integral)
 2025-11-11 12:29:04,689 - [Hilo-ID-1] Inicia procesamiento de
 tarea ID 1 (derivada)
 2025-11-11 12:29:04,690 - [Hilo-ID-2] Inicia procesamiento de
 tarea ID 2 (tesis)
 2025-11-11 12:29:04,691 - [Hilo-ID-3] Inicia procesamiento de
 tarea ID 3 (integral)

2025-11-11 12:29:04,703 - [Hilo-ID-4] Inicia procesamiento de
 tarea ID 4 (integral)
 2025-11-11 12:29:04,723 - [Hilo-ID-5] Inicia procesamiento de
 tarea ID 5 (tesis)
 2025-11-11 12:29:04,733 - [Hilo-ID-1] Tarea completada; TIPO:
 derivada; DURACION: 0.043306s
 2025-11-11 12:29:04,734 - [Hilo-ID-6] Inicia procesamiento de
 tarea ID 6 (derivada)
 2025-11-11 12:29:04,734 - [Hilo-ID-0] Tarea completada; TIPO:
 integral; DURACION: 0.045027s
 2025-11-11 12:29:04,734 - [Hilo-ID-7] Inicia procesamiento de
 tarea ID 7 (integral)
 2025-11-11 12:29:04,736 - [Hilo-ID-8] Inicia procesamiento de
 tarea ID 8 (tesis)
 2025-11-11 12:29:04,736 - [Hilo-ID-6] Tarea completada; TIPO:
 derivada; DURACION: 0.001996s
 2025-11-11 12:29:04,738 - [Hilo-ID-9] Inicia procesamiento de
 tarea ID 9 (tesis)
 2025-11-11 12:29:04,738 - [Hilo-ID-3] Tarea completada; TIPO:
 integral; DURACION: 0.047274s
 2025-11-11 12:29:04,738 - [Hilo-ID-2] Tarea completada; TIPO:
 tesis; DURACION: 0.047274s
 2025-11-11 12:29:04,739 - [Hilo-ID-10] Inicia procesamiento de
 tarea ID 10 (derivada)
 2025-11-11 12:29:04,741 - [Hilo-ID-11] Inicia procesamiento de
 tarea ID 11 (integral)
 2025-11-11 12:29:04,742 - [Hilo-ID-10] Tarea completada; TIPO:
 derivada; DURACION: 0.002990s
 2025-11-11 12:29:04,742 - [Hilo-ID-12] Inicia procesamiento de
 tarea ID 12 (integral)
 2025-11-11 12:29:04,745 - [Hilo-ID-13] Inicia procesamiento de
 tarea ID 13 (tesis)
 2025-11-11 12:29:04,745 - [Hilo-ID-14] Inicia procesamiento de
 tarea ID 14 (derivada)
 2025-11-11 12:29:04,745 - [Hilo-ID-4] Tarea completada; TIPO:
 integral; DURACION: 0.041204s
 2025-11-11 12:29:04,745 - [Hilo-ID-15] Inicia procesamiento de
 tarea ID 15 (tesis)
 2025-11-11 12:29:04,746 - [Hilo-ID-5] Tarea completada; TIPO:
 tesis; DURACION: 0.011443s
 2025-11-11 12:29:04,746 - [Hilo-ID-16] Inicia procesamiento de
 tarea ID 16 (integral)
 2025-11-11 12:29:04,746 - [Hilo-ID-7] Tarea completada; TIPO:
 integral; DURACION: 0.009768s
 2025-11-11 12:29:04,746 - [Hilo-ID-17] Inicia procesamiento de
 tarea ID 17 (tesis)
 2025-11-11 12:29:04,746 - [Hilo-ID-14] Tarea completada; TIPO:
 derivada; DURACION: 0.000997s
 2025-11-11 12:29:04,746 - Se lanzaron 18 hilos. Esperando
 finalizacin (join)...
 2025-11-11 12:29:04,759 - [Hilo-ID-11] Tarea completada; TIPO:
 integral; DURACION: 0.018470s
 2025-11-11 12:29:04,759 - [Hilo-ID-12] Tarea completada; TIPO:
 integral; DURACION: 0.017296s
 2025-11-11 12:29:04,759 - [Hilo-ID-8] Tarea completada; TIPO:
 tesis; DURACION: 0.021265s
 2025-11-11 12:29:04,760 - [Hilo-ID-16] Tarea completada; TIPO:
 integral; DURACION: 0.014304s
 2025-11-11 12:29:04,760 - [Hilo-ID-9] Tarea completada; TIPO:
 tesis; DURACION: 0.022263s
 2025-11-11 12:29:04,761 - [Hilo-ID-13] Tarea completada; TIPO:

tesis; DURACION: 0.015709s
 2025-11-11 12:29:04,761 - [Hilo-ID-15] Tarea completada; TIPO:
 tesis; DURACION: 0.015302s
 2025-11-11 12:29:04,761 - [Hilo-ID-17] Tarea completada; TIPO:
 tesis; DURACION: 0.002263s
 2025-11-11 12:29:04,762 - --- Todas las tareas han sido
 procesadas. ---
 2025-11-11 12:29:04,762 - --- FIN EXPERIMENTO (N=10) ---
 2025-11-11 12:29:04,762 - Tiempo Total: 0.0742s
 2025-11-11 12:29:04,762 - Valor 'largo.txt' (Obtenido): 58

 2025-11-11 12:29:04,762 - --- INICIO EXPERIMENTO (N=20, Con
 Locks) ---
 2025-11-11 12:29:04,763 - Iniciando procesamiento de 18 tareas...
 2025-11-11 12:29:04,763 - [Hilo-ID-0] Inicia procesamiento de
 tarea ID 0 (integral)
 2025-11-11 12:29:04,763 - [Hilo-ID-1] Inicia procesamiento de
 tarea ID 1 (derivada)
 2025-11-11 12:29:04,765 - [Hilo-ID-2] Inicia procesamiento de
 tarea ID 2 (tesis)
 2025-11-11 12:29:04,765 - [Hilo-ID-3] Inicia procesamiento de
 tarea ID 3 (integral)
 2025-11-11 12:29:04,766 - [Hilo-ID-4] Inicia procesamiento de
 tarea ID 4 (integral)
 2025-11-11 12:29:04,773 - [Hilo-ID-1] Tarea completada; TIPO:
 derivada; DURACION: 0.007961s
 2025-11-11 12:29:04,773 - [Hilo-ID-0] Tarea completada; TIPO:
 integral; DURACION: 0.009993s
 2025-11-11 12:29:04,806 - [Hilo-ID-5] Inicia procesamiento de
 tarea ID 5 (tesis)
 2025-11-11 12:29:04,809 - [Hilo-ID-6] Inicia procesamiento de
 tarea ID 6 (derivada)
 2025-11-11 12:29:04,810 - [Hilo-ID-3] Tarea completada; TIPO:
 integral; DURACION: 0.043883s
 2025-11-11 12:29:04,812 - [Hilo-ID-7] Inicia procesamiento de
 tarea ID 7 (integral)
 2025-11-11 12:29:04,813 - [Hilo-ID-8] Inicia procesamiento de
 tarea ID 8 (tesis)
 2025-11-11 12:29:04,813 - [Hilo-ID-9] Inicia procesamiento de
 tarea ID 9 (tesis)
 2025-11-11 12:29:04,815 - [Hilo-ID-4] Tarea completada; TIPO:
 integral; DURACION: 0.036931s
 2025-11-11 12:29:04,816 - [Hilo-ID-10] Inicia procesamiento de
 tarea ID 10 (derivada)
 2025-11-11 12:29:04,816 - [Hilo-ID-6] Tarea completada; TIPO:
 derivada; DURACION: 0.005983s
 2025-11-11 12:29:04,816 - [Hilo-ID-11] Inicia procesamiento de
 tarea ID 11 (integral)
 2025-11-11 12:29:04,818 - [Hilo-ID-12] Inicia procesamiento de
 tarea ID 12 (integral)
 2025-11-11 12:29:04,819 - [Hilo-ID-7] Tarea completada; TIPO:
 integral; DURACION: 0.006020s
 2025-11-11 12:29:04,819 - [Hilo-ID-2] Tarea completada; TIPO:
 tesis; DURACION: 0.052895s
 2025-11-11 12:29:04,822 - [Hilo-ID-13] Inicia procesamiento de
 tarea ID 13 (tesis)
 2025-11-11 12:29:04,823 - [Hilo-ID-14] Inicia procesamiento de
 tarea ID 14 (derivada)
 2025-11-11 12:29:04,824 - [Hilo-ID-15] Inicia procesamiento de
 tarea ID 15 (tesis)
 2025-11-11 12:29:04,824 - [Hilo-ID-10] Tarea completada; TIPO:

derivada; DURACION: 0.008150s
 2025-11-11 12:29:04,824 - [Hilo-ID-11] Tarea completada; TIPO:
 integral; DURACION: 0.006154s
 2025-11-11 12:29:04,825 - [Hilo-ID-16] Inicia procesamiento de
 tarea ID 16 (integral)
 2025-11-11 12:29:04,825 - [Hilo-ID-17] Inicia procesamiento de
 tarea ID 17 (tesis)
 2025-11-11 12:29:04,837 - Se lanzaron 18 hilos. Esperando
 finalizacin (join)...
 2025-11-11 12:29:04,838 - [Hilo-ID-14] Tarea completada; TIPO:
 derivada; DURACION: 0.014962s
 2025-11-11 12:29:04,838 - [Hilo-ID-5] Tarea completada; TIPO:
 tesis; DURACION: 0.027929s
 2025-11-11 12:29:04,839 - [Hilo-ID-12] Tarea completada; TIPO:
 integral; DURACION: 0.019915s
 2025-11-11 12:29:04,839 - [Hilo-ID-16] Tarea completada; TIPO:
 integral; DURACION: 0.013966s
 2025-11-11 12:29:04,839 - [Hilo-ID-8] Tarea completada; TIPO:
 tesis; DURACION: 0.022944s
 2025-11-11 12:29:04,841 - [Hilo-ID-9] Tarea completada; TIPO:
 tesis; DURACION: 0.024529s
 2025-11-11 12:29:04,841 - [Hilo-ID-13] Tarea completada; TIPO:
 tesis; DURACION: 0.018512s
 2025-11-11 12:29:04,842 - [Hilo-ID-15] Tarea completada; TIPO:
 tesis; DURACION: 0.017809s
 2025-11-11 12:29:04,843 - [Hilo-ID-17] Tarea completada; TIPO:
 tesis; DURACION: 0.004062s
 2025-11-11 12:29:04,843 - --- Todas las tareas han sido
 procesadas. ---
 2025-11-11 12:29:04,843 - --- FIN EXPERIMENTO (N=20) ---
 2025-11-11 12:29:04,843 - Tiempo Total: 0.0797s
 2025-11-11 12:29:04,843 - Valor 'largo.txt' (Obtenido): 58

 2025-11-11 12:29:04,843 - --- INICIO EXPERIMENTO (N=50, Con
 Locks) ---
 2025-11-11 12:29:04,844 - Iniciando procesamiento de 18 tareas...
 2025-11-11 12:29:04,845 - [Hilo-ID-0] Inicia procesamiento de
 tarea ID 0 (integral)
 2025-11-11 12:29:04,846 - [Hilo-ID-1] Inicia procesamiento de
 tarea ID 1 (derivada)
 2025-11-11 12:29:04,846 - [Hilo-ID-0] Tarea completada; TIPO:
 integral; DURACION: 0.001494s
 2025-11-11 12:29:04,847 - [Hilo-ID-2] Inicia procesamiento de
 tarea ID 2 (tesis)
 2025-11-11 12:29:04,848 - [Hilo-ID-3] Inicia procesamiento de
 tarea ID 3 (integral)
 2025-11-11 12:29:04,848 - [Hilo-ID-4] Inicia procesamiento de
 tarea ID 4 (integral)
 2025-11-11 12:29:04,848 - [Hilo-ID-1] Tarea completada; TIPO:
 derivada; DURACION: 0.001675s
 2025-11-11 12:29:04,848 - [Hilo-ID-5] Inicia procesamiento de
 tarea ID 5 (tesis)
 2025-11-11 12:29:04,889 - [Hilo-ID-6] Inicia procesamiento de
 tarea ID 6 (derivada)
 2025-11-11 12:29:04,891 - [Hilo-ID-7] Inicia procesamiento de
 tarea ID 7 (integral)
 2025-11-11 12:29:04,894 - [Hilo-ID-6] Tarea completada; TIPO:
 derivada; DURACION: 0.004426s
 2025-11-11 12:29:04,894 - [Hilo-ID-8] Inicia procesamiento de
 tarea ID 8 (tesis)
 2025-11-11 12:29:04,894 - [Hilo-ID-4] Tarea completada; TIPO:

```

integral; DURACION: 0.038153s
2025-11-11 12:29:04,894 - [Hilo-ID-9] Inicia procesamiento de
tarea ID 9 (tesis)
2025-11-11 12:29:04,894 - [Hilo-ID-10] Inicia procesamiento de
tarea ID 10 (derivada)
2025-11-11 12:29:04,896 - [Hilo-ID-11] Inicia procesamiento de
tarea ID 11 (integral)
2025-11-11 12:29:04,896 - [Hilo-ID-3] Tarea completada; TIPO:
integral; DURACION: 0.048397s
2025-11-11 12:29:04,897 - [Hilo-ID-12] Inicia procesamiento de
tarea ID 12 (integral)
2025-11-11 12:29:04,900 - [Hilo-ID-13] Inicia procesamiento de
tarea ID 13 (tesis)
2025-11-11 12:29:04,900 - [Hilo-ID-10] Tarea completada; TIPO:
derivada; DURACION: 0.005159s
2025-11-11 12:29:04,900 - [Hilo-ID-2] Tarea completada; TIPO:
tesis; DURACION: 0.052351s
2025-11-11 12:29:04,900 - [Hilo-ID-7] Tarea completada; TIPO:
integral; DURACION: 0.009148s
2025-11-11 12:29:04,900 - [Hilo-ID-14] Inicia procesamiento de
tarea ID 14 (derivada)
2025-11-11 12:29:04,901 - [Hilo-ID-15] Inicia procesamiento de
tarea ID 15 (tesis)
2025-11-11 12:29:04,901 - [Hilo-ID-16] Inicia procesamiento de
tarea ID 16 (integral)
2025-11-11 12:29:04,902 - [Hilo-ID-11] Tarea completada; TIPO:
integral; DURACION: 0.005777s
2025-11-11 12:29:04,902 - [Hilo-ID-17] Inicia procesamiento de
tarea ID 17 (tesis)
2025-11-11 12:29:04,902 - Se lanzaron 18 hilos. Esperando
finalizacin (join)...
2025-11-11 12:29:04,916 - [Hilo-ID-14] Tarea completada; TIPO:
derivada; DURACION: 0.014372s
2025-11-11 12:29:04,916 - [Hilo-ID-5] Tarea completada; TIPO:
tesis; DURACION: 0.026276s
2025-11-11 12:29:04,916 - [Hilo-ID-12] Tarea completada; TIPO:
integral; DURACION: 0.017951s
2025-11-11 12:29:04,916 - [Hilo-ID-16] Tarea completada; TIPO:
integral; DURACION: 0.012964s
2025-11-11 12:29:04,917 - [Hilo-ID-8] Tarea completada; TIPO:
tesis; DURACION: 0.022938s
2025-11-11 12:29:04,917 - [Hilo-ID-9] Tarea completada; TIPO:
tesis; DURACION: 0.022938s
2025-11-11 12:29:04,918 - [Hilo-ID-13] Tarea completada; TIPO:
tesis; DURACION: 0.017780s
2025-11-11 12:29:04,918 - [Hilo-ID-15] Tarea completada; TIPO:
tesis; DURACION: 0.015957s
2025-11-11 12:29:04,919 - [Hilo-ID-17] Tarea completada; TIPO:
tesis; DURACION: 0.003580s
2025-11-11 12:29:04,919 - --- Todas las tareas han sido
procesadas. ---
2025-11-11 12:29:04,919 - --- FIN EXPERIMENTO (N=50) ---
2025-11-11 12:29:04,919 - Tiempo Total: 0.0751s
2025-11-11 12:29:04,919 - Valor 'largo.txt' (Obtenido): 58

2025-11-11 12:29:04,919 - --- INICIO EXPERIMENTO (N=10, SIN
Locks) ---
2025-11-11 12:29:04,920 - Iniciando procesamiento de 18 tareas...
2025-11-11 12:29:04,920 - [Hilo-ID-0] Inicia procesamiento de
tarea ID 0 (integral)
2025-11-11 12:29:04,920 - [Hilo-ID-1] Inicia procesamiento de

```

tarea ID 1 (derivada)
 2025-11-11 12:29:04,922 - [Hilo-ID-2] Inicia procesamiento de
 tarea ID 2 (tesis)
 2025-11-11 12:29:04,923 - [Hilo-ID-3] Inicia procesamiento de
 tarea ID 3 (integral)
 2025-11-11 12:29:04,939 - [Hilo-ID-4] Inicia procesamiento de
 tarea ID 4 (integral)
 2025-11-11 12:29:04,972 - [Hilo-ID-5] Inicia procesamiento de
 tarea ID 5 (tesis)
 2025-11-11 12:29:04,972 - [Hilo-ID-1] Tarea completada; TIPO:
 derivada; DURACION: 0.050348s
 2025-11-11 12:29:04,972 - [Hilo-ID-0] Tarea completada; TIPO:
 integral; DURACION: 0.052350s
 2025-11-11 12:29:04,973 - [Hilo-ID-6] Inicia procesamiento de
 tarea ID 6 (derivada)
 2025-11-11 12:29:04,973 - [Hilo-ID-3] Tarea completada; TIPO:
 integral; DURACION: 0.049618s
 2025-11-11 12:29:04,973 - [Hilo-ID-7] Inicia procesamiento de
 tarea ID 7 (integral)
 2025-11-11 12:29:04,974 - [Hilo-ID-4] Tarea completada; TIPO:
 integral; DURACION: 0.034904s
 2025-11-11 12:29:04,975 - [Hilo-ID-8] Inicia procesamiento de
 tarea ID 8 (tesis)
 2025-11-11 12:29:04,975 - [Hilo-ID-9] Inicia procesamiento de
 tarea ID 9 (tesis)
 2025-11-11 12:29:04,978 - [Hilo-ID-10] Inicia procesamiento de
 tarea ID 10 (derivada)
 2025-11-11 12:29:04,979 - [Hilo-ID-11] Inicia procesamiento de
 tarea ID 11 (integral)
 2025-11-11 12:29:04,981 - [Hilo-ID-7] Tarea completada; TIPO:
 integral; DURACION: 0.005810s
 2025-11-11 12:29:04,981 - [Hilo-ID-5] Tarea completada; TIPO:
 tesis; DURACION: 0.008732s
 2025-11-11 12:29:04,981 - [Hilo-ID-12] Inicia procesamiento de
 tarea ID 12 (integral)
 2025-11-11 12:29:04,984 - [Hilo-ID-13] Inicia procesamiento de
 tarea ID 13 (tesis)
 2025-11-11 12:29:04,984 - [Hilo-ID-2] Tarea completada; TIPO:
 tesis; DURACION: 0.062071s
 2025-11-11 12:29:04,984 - [Hilo-ID-11] Tarea completada; TIPO:
 integral; DURACION: 0.004987s
 2025-11-11 12:29:04,984 - [Hilo-ID-6] Tarea completada; TIPO:
 derivada; DURACION: 0.009967s
 2025-11-11 12:29:04,985 - [Hilo-ID-14] Inicia procesamiento de
 tarea ID 14 (derivada)
 2025-11-11 12:29:04,986 - [Hilo-ID-9] Tarea completada; TIPO:
 tesis; DURACION: 0.008012s
 2025-11-11 12:29:04,987 - [Hilo-ID-10] Tarea completada; TIPO:
 derivada; DURACION: 0.008977s
 2025-11-11 12:29:04,987 - [Hilo-ID-12] Tarea completada; TIPO:
 integral; DURACION: 0.005984s
 2025-11-11 12:29:04,987 - [Hilo-ID-15] Inicia procesamiento de
 tarea ID 15 (tesis)
 2025-11-11 12:29:04,987 - [Hilo-ID-16] Inicia procesamiento de
 tarea ID 16 (integral)
 2025-11-11 12:29:04,987 - [Hilo-ID-14] Tarea completada; TIPO:
 derivada; DURACION: 0.001995s
 2025-11-11 12:29:04,988 - [Hilo-ID-17] Inicia procesamiento de
 tarea ID 17 (tesis)
 2025-11-11 12:29:05,000 - Se lanzaron 18 hilos. Esperando
 finalizacin (join)...

2025-11-11 12:29:05,000 - [Hilo-ID-8] Tarea completada; TIPO:
tesis; DURACION: 0.021878s
2025-11-11 12:29:05,000 - [Hilo-ID-16] Tarea completada; TIPO:
integral; DURACION: 0.011904s
2025-11-11 12:29:05,001 - [Hilo-ID-13] Tarea completada; TIPO:
tesis; DURACION: 0.016550s
2025-11-11 12:29:05,002 - [Hilo-ID-15] Tarea completada; TIPO:
tesis; DURACION: 0.013559s
2025-11-11 12:29:05,002 - [Hilo-ID-17] Tarea completada; TIPO:
tesis; DURACION: 0.001656s
2025-11-11 12:29:05,003 - --- Todas las tareas han sido
procesadas. ---
2025-11-11 12:29:05,003 - --- FIN EXPERIMENTO (SIN Locks) ---
2025-11-11 12:29:05,003 - Tiempo Total: 0.0826s
2025-11-11 12:29:05,003 - Valor 'largo.txt' (Obtenido): 43
(Probablemente incorrecto)

Fuente: autoria propia.